

Tic-Tac-Toe Game on Arduino for Beginners



By IEEE BITS Pilani

Published June 17, 2023 on Instructables.com



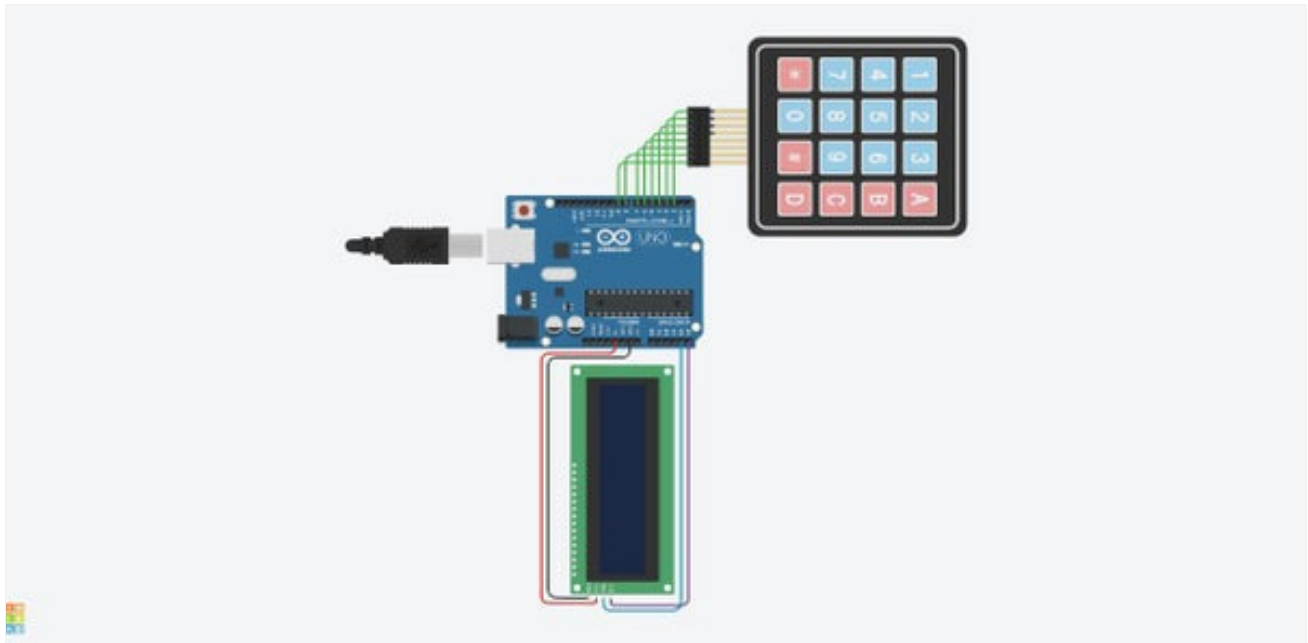
Introducing a fun and interactive project made on Arduino – tic-tac-toe! This classic game is brought to life using an Arduino board, keypad buttons, and an LCD display. By connecting buttons to the Arduino, players can make their moves on a 3x3 grid displayed on the LCD. The Arduino board handles the game logic, detecting button presses, updating the game state, and checking for win conditions. The LCD display provides real-time feedback, showing the game board and current player's turn. With a straightforward hardware setup and intuitive gameplay, this tic-tac-toe project on Arduino offers an enjoyable and engaging experience for players of all ages.

Supplies

- Arduino UNO Board
- Keypad buttons
- I2C LCD (20x4) [You can use a smaller LCD display if you don't have a 20x4]
- Jumper Wires
- Breadboard
- Some libraries (mentioned in step 2 along with links)

If you don't have the components, you can simulate it on [TinkerCAD](https://www.tinkercad.com/circuits) (<https://www.tinkercad.com/circuits>) to learn the basics as well.

Step 1: Connection Schematics



Keypad :

1 to 8 -> D2 to D9

LCD (With I2C) :

SDA Pin -> A4 Pin

SCL Pin -> A5 Pin

VCC Pin -> 5V

GND Pin -> GND

(Even if you are on TinkerCAD, similar connections need to be done)

Step 2: Code

After making the connections, open up the Arduino IDE:

- [Download](https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE) (<https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE>)
- [Web Editor](https://create.arduino.cc/editor) (<https://create.arduino.cc/editor>)

Download the following libraries if doing on your local machine:

- [Keypad](https://github.com/Chris--A/Keypad) (<https://github.com/Chris--A/Keypad>)
- [I2C LCD](https://reference.arduino.cc/reference/en/libraries/liquidcrystal-i2c/) (<https://reference.arduino.cc/reference/en/libraries/liquidcrystal-i2c/>)

Copy-paste the code. You can understand this by reading the comments and see how the algorithm is working.

```
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>

// LCD Settings
LiquidCrystal_I2C lcd(0x27, 20, 4); // Change the address if necessary

// Game Constants
const int BUTTON_ROWS = 3;
const int BUTTON_COLS = 3;

// Keypad constants
const byte ROWS = 4;
const byte COLS = 4;

// Array to represent keys on keypad
char hexaKeys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

// Connections to the arduino
byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3, 2};

// Create keypad object
Keypad customKeypad = Keypad(makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS);

// Player Constants
const int PLAYER_NONE = 0;
const int PLAYER_X = 1;
const int PLAYER_O = 2;

// Game Variables
int board[BUTTON_ROWS][BUTTON_COLS];
int currentPlayer;
bool gameOver;

void setup() {
  // Initialize LCD Display
  lcd.begin(20, 4);
  lcd.backlight();
  lcd.print("Tic-Tac-Toe");
  delay(2000);
  lcd.clear();

  // Initialize Game Variables
  currentPlayer = PLAYER_X;
  gameOver = false;
  resetBoard();
  updateDisplay();
}

void loop() {
  // Check for Button Press
  char keypress = customKeypad.getKey();

  if (keypress == '0') {
    setup();
  }

  if (!gameOver) {
    if (keypress <= 57 && keypress >= 49) {
      int number = keypress - 57 + 9;

      int i = (number - 1) / 3;
      int j = ((number - 1) % 3);

      if (board[i][j] == PLAYER_NONE) {
        board[i][j] = currentPlayer;
        updateDisplay();
      }
    }
  }
}
```

```

        checkWinCondition();
        switchPlayers();
    }
    delay(200); // Button Rebound
}
}
}

void resetBoard() {
    for (int i = 0; i < BUTTON_ROWS; i++) {
        for (int j = 0; j < BUTTON_COLS; j++) {
            board[i][j] = PLAYER_NONE;
        }
    }
}

void updateDisplay() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Player ");
    lcd.print(currentPlayer == PLAYER_X ? "X" : "O");
    lcd.setCursor(0, 1);
    delay(1);
    for (int i = 0; i < BUTTON_ROWS; i++) {
        lcd.setCursor(0, i + 1);
        for (int j = 0; j < BUTTON_COLS; j++) {
            lcd.print("|");
            lcd.print(symbolForPlayer(board[i][j]));
        }
        lcd.print("|");
    }
    // lcd.setCursor(0, 5);
    // lcd.print("-----");
}

char symbolForPlayer(int player) {
    if (player == PLAYER_X) {
        return 'X';
    } else if (player == PLAYER_O) {
        return 'O';
    }
    return ' ';
}

void switchPlayers() {
    currentPlayer = (currentPlayer == PLAYER_X) ? PLAYER_O : PLAYER_X;
}

void checkWinCondition() {
    // Check Rows
    for (int i = 0; i < BUTTON_ROWS; i++) {
        if (board[i][0] != PLAYER_NONE && board[i][0] == board[i][1] && board[i][1] == board[i][2]) {
            gameOver = true;
            lcd.setCursor(0, 4);
            lcd.print("Player ");
            lcd.print(symbolForPlayer(board[i][0]));
            lcd.print(" wins!");
            return;
        }
    }

    // Check Columns
    for (int j = 0; j < BUTTON_COLS; j++) {
        if (board[0][j] != PLAYER_NONE && board[0][j] == board[1][j] && board[1][j] == board[2][j]) {
            gameOver = true;
            lcd.setCursor(0, 4);
            lcd.print("Player ");
            lcd.print(symbolForPlayer(board[0][j]));
            lcd.print(" wins!");
            return;
        }
    }
}

```

```
// Check Diagonals
if (board[0][0] != PLAYER_NONE && board[0][0] == board[1][1] && board[1][1] == board[2][2]) {
  gameOver = true;
  lcd.setCursor(0, 4);
  lcd.print("Player ");
  lcd.print(symbolForPlayer(board[0][0]));
  lcd.print(" wins!");
  return;
}

if (board[0][2] != PLAYER_NONE && board[0][2] == board[1][1] && board[1][1] == board[2][0]) {
  gameOver = true;
  lcd.setCursor(0, 4);
  lcd.print("Player ");
  lcd.print(symbolForPlayer(board[0][2]));
  lcd.print(" wins!");
  return;
}

// Check Draw
bool draw = true;
for (int i = 0; i < BUTTON_ROWS; i++) {
  for (int j = 0; j < BUTTON_COLS; j++) {
    if (board[i][j] == PLAYER_NONE) {
      draw = false;
      break;
    }
  }
  if (!draw) {
    break;
  }
}
if (draw) {
  gameOver = true;
  lcd.setCursor(0, 4);
  lcd.print("It's a draw!");
}
}
```

Step 3: Explanation

1. The code begins by including the necessary libraries and defining constants for the game grid, player symbols, and keypad buttons. These constants make it easier to modify the code if needed. Keypad buttons are calibrated to represent the grid of tic-tac-toe. 0 Key is used as reset
2. The setup function is called once at the beginning of the program. It initializes the LCD display, button pins, and game variables. This ensures that everything is set up correctly before the game starts.
3. The loop function is the main part of the code and runs continuously. It checks for button presses and handles the logic of the tic-tac-toe game.
4. The resetBoard function is a helper function that resets the game board to its initial state. This is useful when the game needs to be restarted.
5. The updateDisplay function is another helper function that updates the LCD display with the current game state. It ensures that the players can see the game board and any relevant messages during gameplay.
6. The symbolForPlayer function is used to determine the symbol ('X' or 'O') corresponding to the current player. This is helpful when updating the game board with the player's moves.
7. The switchPlayers function is responsible for alternating between players after each move. It keeps track of the current player and switches to the other player's turn.
8. The checkWinCondition function checks if a player has won the game or if it is a draw. It examines the game board to see if there are any winning combinations or if all the squares are filled. If a win or draw condition is detected, the game is ended.
9. The game logic and LCD updates work together to provide an interactive tic-tac-toe game on the Arduino. The buttons allow players to make their moves, the LCD display shows the game board and messages, and the code handles the game logic and win conditions.
10. As an addition, You can consider adding LEDs, Buzzers and other

interactive components to make the project more aesthetic and attractive.